

Consideraciones generales

Variables principales

Realizar una tabla indicando:

Variable	¿Qué almacena?	Tipo de dato
nombre_cliente	Nombre del cliente	str
edad_cliente	Edad	int
total_compra	Total de la compra	float

Estructuras utilizadas

Explicar:

- ¿Dónde utilizaron listas?
- ¿Dónde utilizaron diccionarios?
- ¿Para qué sirve cada una?
- ¿Por qué eligieron esas estructuras?

Condicionales

Indicar:

- ¿Dónde utilizaron if?
- ¿Dónde utilizaron elif?
- ¿Dónde utilizaron else?
- ¿Qué problema resuelve cada condición?

Ciclos

Explicar:

- ¿Dónde utilizaron while?
 - ¿Dónde utilizaron for?
 - ¿Qué función cumple cada ciclo?
10. Funciones

Realizar una lista de las funciones creadas.

Por ejemplo:

```
mostrar_menu()  
agregar_producto()  
buscar_producto()  
registrar_venta()  
mostrar_estadisticas()
```

Y explicar brevemente qué realiza cada una.

DISEÑO DEL ALGORITMO

Antes de presentar el programa terminado deberán diseñar el algoritmo.

Pueden utilizar:

Pseudocódigo
Diagrama de flujo

Secuencia de pasos

El algoritmo debe representar realmente cómo funciona el programa.

No se aceptará un diagrama que no coincida con el código final.

La evaluación contempla específicamente que el algoritmo sea claro, estructurado y coherente con el problema.

8. CÓDIGO FUENTE

El código deberá:

Estar correctamente indentado.

Ser legible.

Utilizar nombres descriptivos.

Estar organizado mediante funciones.

Evitar código innecesario o repetido.

Contener comentarios útiles.

Ejecutarse correctamente.

Tener un menú funcional.

La presentación, organización, indentación y utilización de comentarios forman parte de los criterios de evaluación.

9. PRUEBAS DEL PROGRAMA

Deberán realizar diferentes pruebas antes de entregar el proyecto.

Como mínimo deberán mostrar:

Prueba 1

Ingreso de datos correctos.

Prueba 2

Ingreso de datos incorrectos.

Prueba 3

Utilización de una opción incorrecta del menú.

Prueba 4

Búsqueda de un dato que existe.

Prueba 5

Búsqueda de un dato que no existe.

Prueba 6

Una situación límite relacionada con su proyecto.

Para cada prueba deberán indicar:

- ¿Qué ingresamos?
- ¿Qué esperábamos que sucediera?
- ¿Qué sucedió?
- ¿El resultado fue correcto?

10. PRESENTACIÓN FINAL

Cada pareja deberá entregar:

INFORME

Con:

- Portada.
- Problemática.
- Objetivo.
- Propuesta de solución.
- Funcionamiento.
- Variables.
- Estructuras utilizadas.
- Funciones.
- Algoritmo.
- Pruebas.
- Conclusión.

PROGRAMA

Archivo .py funcionando correctamente.

DEFENSA ORAL

Ambos integrantes deberán conocer y comprender todo el proyecto.

No alcanza con conocer solamente la parte del código que realizó cada integrante.

La evaluación de UTN establece que los integrantes presentes deben poder explicar y justificar las consultas realizadas durante el coloquio.

11. COLOQUIO / DEFENSA

Durante la defensa podrán realizarse preguntas como:

Sobre el problema

- ¿Qué problema intenta resolver el programa?
- ¿Por qué eligieron esta problemática?
- ¿Quién utilizaría este programa?
- ¿Cómo su propuesta soluciona el problema?

Sobre el algoritmo

Expliquen el funcionamiento del programa desde que comienza hasta que finaliza.

- ¿Qué sucede cuando el usuario selecciona una opción del menú?
- ¿Dónde utilizaron condiciones?
- ¿Dónde utilizaron ciclos?

Sobre las variables

- ¿Qué tipo de dato almacena esta variable?

¿Por qué utilizaron int, float o str?
 ¿Qué sucedería si cambiamos el tipo de dato?
 Sobre listas y diccionarios
 ¿Qué información almacena su lista?
 ¿Qué información almacena su diccionario?
 ¿Por qué utilizaron una lista y no otra estructura?
 ¿Cómo acceden a un dato almacenado?
 Sobre funciones
 ¿Por qué dividieron el programa en funciones?
 ¿Qué hace esta función?
 ¿Qué información recibe?
 ¿Qué devuelve mediante return?
 Sobre operadores
 ¿Dónde utilizaron ==?
 ¿Dónde utilizaron !=?
 ¿Dónde utilizaron and?
 ¿Dónde utilizaron or?
 ¿Qué operación matemática realiza esta parte del código?
 Sobre ciclos
 ¿Por qué utilizaron while?
 ¿Por qué utilizaron for?
 ¿Qué condición hace que termine el while?
 ¿Qué información recorre el for?
 Sobre errores
 ¿Qué ocurre si el usuario ingresa un dato incorrecto?
 ¿Qué validaciones realizaron?
 ¿Qué parte del programa fue la más difícil de resolver?
 ¿Qué mejorarían si tuvieran más tiempo?

12. DESAFÍO DURANTE LA DEFENSA

Durante el coloquio, el docente podrá solicitar pequeñas modificaciones o situaciones hipotéticas.

Por ejemplo:

"Quiero que ahora el programa muestre solamente los productos que tienen menos de 5 unidades."

O:

"Quiero que muestre solamente los libros disponibles."

O:

"Quiero agregar una opción al menú que muestre cuántos elementos hay almacenados."

La finalidad de esta actividad será comprobar que los integrantes comprenden realmente el código y no solamente lo presentan funcionando.

13. CRITERIOS GENERALES DE EVALUACIÓN

Se tendrá en cuenta:

Aspecto	Se evaluará
Problemática	Claridad del problema planteado

Propuesta Que el programa resuelva el problema
Documentación Que esté completa y correctamente desarrollada
Algoritmo Claridad y coherencia
Menú Funcionamiento y facilidad de uso
Condicionales Uso correcto
Ciclos Uso correcto de while y for
Variables Nombres descriptivos y uso adecuado
Operadores Uso de operadores lógicos, matemáticos y relacionales
Listas Uso correcto y con propósito
Diccionarios Uso correcto y con propósito
Funciones Organización y reutilización del código
Código Orden, indentación y legibilidad
Comentarios Claros y útiles
Funcionamiento Programa ejecutable y sin errores
Defensa Comprensión y explicación del proyecto
Estos criterios se corresponden con los aspectos contemplados en la rúbrica de Python
Fundamentos utilizada para la evaluación.
14. IMPORTANTE

El objetivo de este trabajo no es solamente entregar un programa que funcione.

El objetivo es demostrar que pueden:

ANALIZAR → DISEÑAR → PROGRAMAR → PROBAR → EXPLICAR

Un programa es una solución informática formada por instrucciones y algoritmos que permiten resolver una problemática.

Por este motivo, ambos integrantes deben comprender el funcionamiento completo del proyecto.

15. ENTREGA FINAL – CHECKLIST

Antes de entregar, controlar:

- ☐ Tenemos definida una problemática.
- ☐ Tenemos definido el objetivo.
- ☐ Explicamos nuestra propuesta de solución.
- ☐ Tenemos un menú funcional.
- ☐ Utilizamos if / elif / else.
- ☐ Utilizamos while.
- ☐ Utilizamos for.
- ☐ Utilizamos una lista.
- ☐ Utilizamos un diccionario.

- ☐ Utilizamos funciones creadas con def.
- ☐ Utilizamos operadores matemáticos.
- ☐ Utilizamos operadores relacionales.
- ☐ Utilizamos operadores lógicos.
- ☐ Validamos los datos ingresados.
- ☐ Las variables tienen nombres descriptivos.
- ☐ El código está correctamente indentado.
- ☐ Agregamos comentarios útiles.
- ☐ Tenemos pseudocódigo/diagrama de flujo/secuencia de pasos.
- ☐ Probamos el programa con diferentes situaciones.
- ☐ El programa funciona correctamente.
- ☐ Ambos integrantes conocen todo el código.
- ☐ Ambos integrantes pueden explicar las decisiones tomadas.

¡A PROGRAMAR!

Recuerden: no se evalúa solamente cuánto código escribieron, sino qué problema resolvieron, cómo lo resolvieron y si pueden explicar por qué su programa funciona de esa manera.